BACSE202

DATABASE SYSTEMS PROJECT
FINAL REPORT

CRIMINAL CASE MANAGEMENT & CRIME ANALYTICS SYSTEM

Project Explanation Video: [INSERT VIDEO LINK]



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

VIT CHENNAI
School of Computer Science and Engineering
Vellore Institute of Technology, Chennai
October 2026

A project report on

CRIMINAL CASE MANAGEMENT & CRIME ANALYTICS SYSTEM

Submitted in partial fulfilment of the requirements for the course
BACSE202 – DATABASE SYSTEMS

Bachelor of Technology in Artificial Intelligence and Machine Learning

Submitted by:

Ayush Sharma [25BAI1392]

Ojas Sugur [25BAI1222]



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCOPE
School of Computer Science and Engineering
Vellore Institute of Technology, Chennai
October 2026

DECLARATION

We hereby declare that the project report entitled "CRIMINAL CASE MANAGEMENT & CRIME ANALYTICS SYSTEM" submitted by Ayush Sharma [25BAI1392] and Ojas Sugur [25BAI1222] for the course of BACSE202 DATABASE SYSTEMS, School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Chennai, is a record of original work carried out by us under the supervision of DR. HEMALATHA K (Assistant Professor Senior Grade 1) [or Faculty Project Supervisor].

We further declare that the work reported in this project has not been submitted, either in part or in full, for the award of any other degree, diploma, or academic credit at this Institute or any other institute or university.

The technical descriptions, database schemas, normalization proofs, SQL scripts, stored routines, test benchmarks, and system screenshots in this report have been prepared with reference to our implemented project repository and its accompanying documentation. Results reported have been verified as authentic system evidence against our live MySQL Community Server 9.7 database instance and automated test suite.

Place: VIT CHENNAI
Date: 30/10/2026

Ayush Sharma [25BAI1392]
Ojas Sugur [25BAI1222]

SCOPE
School of Computer Science and Engineering

CERTIFICATE

This is to certify that the project report entitled "CRIMINAL CASE MANAGEMENT & CRIME ANALYTICS SYSTEM" is prepared and submitted by Ayush Sharma [25BAI1392] and Ojas Sugur [25BAI1222] to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirements for the award of the course project of BACSE202 DATABASE SYSTEMS. The project report has been prepared under my guidance and, subject to review and approval by the appropriate academic authorities, is submitted for evaluation.

The candidates have ensured that all database architecture specifications, Boyce-Codd Normal Form (BCNF) proofs, multi-level security access control models, benchmark counts, and system limitations are accurately represented in this final submitted version.

Signature of Supervisor: _____

Name: DR. HEMALATHA K [or Faculty Supervisor Placeholder]
Designation: Assistant Professor Senior Grade 1
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology, Chennai
Date: 30/10/2026

ABSTRACT

The Criminal Case Management & Crime Analytics System (CCMS) is an enterprise-grade relational database and analytical platform engineered to overcome the critical operational shortcomings of paper-based general diaries and fragmented legacy police dockets. Traditional precinct workflows suffer from data duplication, misplaced case files, unverified custody tracking, and dangerous over-privileged database accounts. To resolve these challenges, our team developed a mathematically normalized 14-table relational database strictly satisfying Boyce-Codd Normal Form (BCNF) on MySQL Community Server 9.7 (InnoDB Storage Engine), paired with a responsive Python 3.14 Flask web application and interactive Chart.js dashboards.

The application provides end-to-end criminal dossier lifecycle tracking, person registries for suspects, victims, and witnesses, and an evidence catalog with automated, trigger-enforced chain-of-custody logging. To bridge public safety and community transparency safely, the architecture introduces a decoupled Citizen Portal where victims can file non-emergency reports, receive automated tracking references (CR-YYYY-XXXXX), and monitor status updates without exposing confidential investigative intelligence. An anti-enumeration defense returning HTTP 404 Not Found prevents unauthorized tracking ID probing. The platform also ingests over 50,000 public crime records from the City of Chicago Socrata Open Data Portal, reconciling incidents with canonical Illinois Uniform Crime Reporting (IUCR) lookup codes.

Operational security is enforced through a dual-account least-privilege model isolating migration authority (crimedb_admin) from web runtime operations (crimedb_app), combined with Bell-LaPadula Multi-Level Security across four hierarchical tiers (PUBLIC < INTERNAL < CONFIDENTIAL < RESTRICTED) and discretionary Need-to-Know assignment checks. The system has been validated through 35 automated Pytest integration tests and a 24-point end-to-end smoke test, achieving 100% pass rates. Secondary composite B-tree indexing reduced examined query rows by over 95% on EXPLAIN benchmarks. Current operational boundaries are focused on academic demonstration, with future scope targeting GIS spatial indexing, biometric vector search, and blockchain-anchored custody ledgers.

Keywords: Criminal case management, relational database, MySQL 9.7, Boyce-Codd Normal Form, Bell-LaPadula security, chain of custody, citizen portal, crime analytics, Flask, least privilege.

ACKNOWLEDGEMENT

We express our sincere gratitude to DR. HEMALATHA K (Assistant Professor Senior Grade 1), School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Chennai, for guidance, constructive feedback, and academic encouragement during the conceptualization, implementation, and documentation of this project report.

We thank the faculty members and project coordinators of VIT Chennai and SCOPE for providing an academic environment in which modern relational database management, multi-level security, and web software engineering can be explored and synthesized. We are also grateful to the developers and maintainers of the open-source technologies utilized in this system, including Python, Flask, SQLAlchemy, MySQL, Bootstrap, and Chart.js, as well as the City of Chicago Department of Innovation and Technology for providing the public Socrata Open Data Portal API.

We would also like to thank each other as project teammates for seamless collaboration throughout the design, testing, and debugging phases. Finally, we acknowledge that an enterprise law enforcement record system can only be evaluated responsibly when its operational boundaries and academic demonstration constraints are stated clearly; this report therefore carefully distinguishes verified local implementation results from future production extensions.

TABLE OF CONTENTS

Title / Section	Page
ABSTRACT	iii
ACKNOWLEDGEMENT	iv
LIST OF TABLES	vi
LIST OF FIGURES	vii
LIST OF ABBREVIATIONS AND NOMENCLATURE	viii
CHAPTER 1: PROBLEM STATEMENT	1
1.1 Operational Deficiencies of Manual Registers	1
1.2 Relational Anomalies & Data Corruption	2
1.3 Privilege Over-Granting & Security Risks	2
CHAPTER 2: PROJECT OBJECTIVES & SCOPE	3
2.1 Mathematical Normalization (BCNF)	3
2.2 Multi-Level Security & Need-to-Know	3
2.3 Dual-Account Least Privilege Protection	4
2.4 Decoupled Citizen Portal	4

Title / Section	Page
CHAPTER 3: LITERATURE SURVEY & EXISTING SYSTEMS	5
3.1 Prior Research in Digital Policing Systems	5
3.2 Theoretical Database & Security Foundations	6
CHAPTER 4: SYSTEM REQUIREMENTS & USER ROLES	7
4.1 User Roles & Clearance Hierarchy	7
4.2 Functional Requirements (FR-1 to FR-12)	8
4.3 Non-Functional Requirements (NFR-1 to NFR-5)	9
CHAPTER 5: SYSTEM DESIGN & ARCHITECTURE	10
5.1 Layered Presentation, Logic & Storage Architecture	10
5.2 Dual-Account Least-Privilege Architecture	11
5.3 Bell-LaPadula Confidentiality Model	12
CHAPTER 6: DATABASE DESIGN, ER & EER MODELING	14
6.1 Enhanced Entity-Relationship (EER) Specialization	14

Title / Section	Page
6.2 Complete 14-Table Relational ER Diagram	15
CHAPTER 7: RELATIONAL SCHEMA & DATA DICTIONARY	17
CHAPTER 8: FUNCTIONAL DEPENDENCIES & BCNF PROOFS	22
CHAPTER 9: SQL IMPLEMENTATION & QUERY CATALOG	25
CHAPTER 10: ADVANCED DATABASE FEATURES	29
CHAPTER 11: CLOUD DEPLOYMENT ARCHITECTURE	34
CHAPTER 12: TESTING & QUALITY ASSURANCE	36
CHAPTER 13: PERFORMANCE ANALYSIS & INDEX OPTIMIZATION	39
CHAPTER 14: RESULTS & DISCUSSION	41
CHAPTER 15: CONCLUSION	43
CHAPTER 16: FUTURE SCOPE	44
CHAPTER 17: REFERENCES	45

LIST OF TABLES

Table Caption	Page
Table 3.1. Comparison of prior policing systems and current CCMS architecture	5
Table 4.1. User roles, clearance tiers, demo credentials, and responsibilities	7
Table 5.1. Database account privilege segregation and operational prohibitions	11
Table 7.1. Comprehensive Data Dictionary across all 14 BCNF tables	18
Table 12.1. Automated test suite execution matrix (35/35 passing)	36
Table 13.1. Composite B-tree secondary index formulation and target query paths	39
Table 13.2. EXPLAIN benchmark comparison before and after composite indexing	40

LIST OF FIGURES

Figure Caption	Page
Figure 5.1. Multi-tier application and database architecture	10
Figure 6.1. Enhanced Entity-Relationship (EER) model showing personnel specialization	14
Figure 6.2. Complete 14-table Relational Entity-Relationship Diagram (ERD)	16
Figure 11.1. Cloud production deployment topology with VPC and managed RDS	35

LIST OF ABBREVIATIONS AND NOMENCLATURE

Abbreviation / Term	Meaning & Context
API	Application Programming Interface
ACID	Atomicity, Consistency, Isolation, and Durability
BCNF	Boyce-Codd Normal Form
CCMS	Cyber-Crime & Criminal Case Management System
CK	Candidate Key
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DDL	Data Definition Language
DML	Data Manipulation Language
EER	Enhanced Entity-Relationship
ERD	Entity-Relationship Diagram
ETL	Extract, Transform, Load
FIR	First Information Report
FK	Foreign Key
IUCR	Illinois Uniform Crime Reporting
JSON	JavaScript Object Notation

Abbreviation / Term	Meaning & Context
MLS	Multi-Level Security
NFR	Non-Functional Requirement
PK	Primary Key
RBAC	Role-Based Access Control

Chapter 1: Problem Statement

Modern police stations handle an increasing volume of complex offenses, ranging from physical burglaries and assaults to cyber-enabled bank fraud, ransomware, and identity theft. Despite technological advancements in consumer software, many municipal precincts and rural police desks still rely heavily on paper general diaries and fragmented log registers. During our initial analysis in Digital Assignment 1 (DA1), our team identified six practical failure points in existing manual and legacy systems:

1.1 Manual Inefficiencies & Lost Records: Manual FIR registers require officers to write down complainant details, suspect descriptions, and stolen property repeatedly. If a physical dossier is misplaced or damaged, weeks of investigative progress vanish before forensic inquiries can even start.

1.2 Normalization Failures & Data Anomalies: Earlier digital attempts often store officer ranks, precinct telephone numbers, or offense descriptions redundantly across every incident row. This causes update anomalies when an officer is reassigned, deletion anomalies when an old incident is purged, and insertion anomalies when an officer cannot be registered without an active case.

1.3 Tamperable Chain-of-Custody: In standard record systems, evidence custody entries are maintained in generic, editable tables. A compromised staff account or dishonest actor could retroactively edit evidence locker locations, delete transfer logs, or alter seized quantities without leaving a forensic audit trail.

1.4 Over-Privileged Database Connections: Most educational and entry-level web applications connect to MySQL using an administrative account with blanket privileges (GRANT ALL). An SQL injection vulnerability or coding bug in such a setup allows attackers to drop core tables or view unredacted intelligence.

1.5 Lack of Access Classification & Need-to-Know: In traditional setups, any logged-in officer can browse any case in the precinct. This violates basic operational security, because sensitive homicide files, internal affairs inquiries, or confidential informant reports must never be visible to general patrol officers or unassigned detectives.

1.6 Insecure Citizen Engagement: Victims filing complaints have no transparent mechanism to monitor case progress short of visiting the station in person. Conversely, directly connecting public web forms to internal police dossiers risks data pollution, SQL injection, and enumeration attacks where attackers probe tracking numbers to deduce confidential filings.

Chapter 2: Project Objectives & Scope

To systematically address these issues, our team established clear, achievable engineering objectives that guided both our database design and application development:

2.1 Relational Rigor (BCNF): Design a 14-table relational schema that satisfies Boyce-Codd Normal Form across all entities, ensuring every non-trivial functional dependency has a candidate key as its determinant.

2.2 Security Clearance Hierarchy: Model four hierarchical clearance and classification levels (PUBLIC < INTERNAL < CONFIDENTIAL < RESTRICTED) adhering to the Bell-LaPadula Simple Security Property ('No Read Up'), supplemented by active assignment verification for Need-to-Know access.

2.3 Engine Privilege Isolation: Configure two distinct MySQL accounts: crimedb_admin for schema DDL and DEFINER triggers, and crimedb_app for runtime web queries with zero DDL privileges, zero direct write access to audit/custody ledgers, and zero delete access on citizen accounts.

2.4 Public Service & Citizen Privacy: Implement an isolated citizen workflow (/citizen/*) with independent session storage, salted scrypt password hashing, automated tracking reference generation (CR-YYYY-XXXXX), and an anti-enumeration HTTP 404 security response.

2.5 External Data Reconciliation: Connect the system to the City of Chicago Socrata Open Data Portal, ingesting and normalizing real crime records into public_crime_incidents while mapping them to canonical IUCR lookup codes.

2.6 Server-Side Database Routines: Implement atomic stored procedures (sp_assign_case), deterministic stored functions (fn_case_duration), DEFINER database triggers, analytical views, and declarative virtual generated keys.

2.7 Performance Benchmarking: Formulate composite secondary B-tree indexes targeting frequent query paths and benchmark query execution using EXPLAIN to prove substantial reductions in examined rows.

Chapter 3: Literature Survey & Existing Systems

During the literature review phase in DA1, Ojas Sugur reviewed prior research on digital policing, case tracking, and crime analytics. We expanded that analysis during the final implementation to examine database theory and multi-level security models:

Table 3.1. Comparison of prior policing systems and current CCMS architecture

Study & Authors	Architecture / Tech	Key Contributions	Observed Limitations
Sharma et al. (2022)	PHP + MySQL Web Application	Role-Based Access Control; reduced police paperwork by ~65%.	Lacked cybercrime specifics (IP addresses, transaction logs) and multi-victim M:N handling.
Adebayo & Oladipo (2021)	Java Swing + MySQL (JDBC)	Enforced foreign key constraints to eliminate duplicate arrest records.	Heavy graphical desktop client ran poorly on low-spec precinct computers; lacked web access.
Chen & Williams (2023)	PostgreSQL Relational DB	Linked financial fraud losses directly to suspect profiles.	Highly complex and expensive administration; non-technical duty officers struggled with day-to-day operation.
Gupta et al. (2020)	Java CLI + SQLite	Extremely fast execution on weak hardware; simplified FIR logging.	SQLite locks the whole database during writes, preventing concurrent multi-user network access.

Study & Authors	Architecture / Tech	Key Contributions	Observed Limitations
Fernandez & Miller (2022)	Data Mining & 3NF SQL	Clean 3NF relational normalization for statistical crime reporting.	Purely analytical batch system; had no transactional interface for entering live daily complaints.

In addition to these applied policing papers, our team drew foundational concepts from classic database and security literature: Bell and LaPadula's seminal work (1973) provided the formal basis for our Multi-Level Security clearance ranks; Elmasri and Navathe's Database Systems (2015) guided our Boyce-Codd Normal Form dependency analysis; and Silberschatz et al. (2020) supplied the concurrency and transaction control patterns used in our stored routines and triggers.

Chapter 4: System Requirements & User Roles

Our system defines four distinct user roles, each operating with specific business responsibilities and database privileges:

Table 4.1. User roles, clearance tiers, demo credentials, and responsibilities

User Role	Demo Username / Email	Assigned Clearance	Operational Responsibilities
System Administrator	admin_sys	RESTRICTED (Level 4)	Chief Eleanor Vance: Department-wide operational oversight, staff clearance management (/admin/staff), trigger audit ledger review, and case triage.
Senior Detective	investigator_chen	CONFIDENTIAL (Level 3)	Detective Chen: Assigned lead investigations, confidential diary notes, evidence custody transfers, and suspect interview memos.
Patrol Officer	officer_carter	INTERNAL (Level 2)	Officer Carter: Initial case docket intake, witness registration, evidence intake. Cannot access classified confidential/restricted files.

User Role	Demo Username / Email	Assigned Clearance	Operational Responsibilities
Public Citizen	citizen.jane@example.com	PUBLIC (Level 1)	Jane Doe: Separate public citizen registration, non-emergency report submission (CR-YYYY-XXXXX), and tracking public progress updates.

Functional Requirements Specification

• FR-1 (Authentication & Password Security): Secure user authentication with salted scrypt hashing (scrypt:32768:8:1) for both staff and citizens.

• FR-2 (Multi-Level Security Hierarchy): Strict database and application filtering based on PUBLIC (1), INTERNAL (2), CONFIDENTIAL (3), and RESTRICTED (4).

• FR-3 (Need-to-Know Verification): Blocking unassigned investigators from CONFIDENTIAL and RESTRICTED cases with HTTP 403 Forbidden.

• FR-4 (Admin Staff Governance): Interactive clearance management panel (/admin/staff) to elevate/demote personnel clearance and toggle account status.

• FR-5 (Case Dossier CRUD): Complete lifecycle management of cases, tracking incident locations, priorities, categories, and closure dates.

• FR-6 (Automated Assignment Routine): Atomic stored procedure (sp_assign_case) ensuring exactly one active Lead Investigator per case.

• FR-7 (Evidence Custody Ledger): Tracking physical items with barcodes; immutable transfer logs automatically recorded by DEFINER triggers.

• FR-8 (Investigation Diary Notes): Attaching progress notes to cases, with confidentiality flags restricting sensitive entries to assigned staff.

• FR-9 (Chicago Open Crimes Analytics): Querying and visualizing over 50,000 live Socrata crime incidents with district, year, and type filters.

• FR-10 (Citizen Portal Intake): Decoupled citizen registration and report filing generating unique public tracking references (CR-YYYY-XXXXX).

• FR-11 (Anti-Enumeration Privacy Defense): Responding with HTTP 404 Not Found (instead of 403) when a citizen probes another citizen's report.

• FR-12 (SQL Evaluation Workbench): Interactive /sql-demo console allowing evaluators to run queries, view views, execute routines, and inspect EXPLAIN plans.

Chapter 5: System Design & Architecture

The application follows a modular three-tier architecture separating presentation, business security policies, and physical MySQL storage:

Presentation Tier: Responsive web templates built with Bootstrap 5.3.3 and Chart.js 4.4.1. Clean interface navigation ensures staff users only see modules permitted by their clearance, while public citizens interact through a dedicated portal theme with emergency disclaimers.

Application & Security Tier: Built using Python 3.14 and Flask 3.1.2. The security service (security_service.py) enforces Bell-LaPadula clearance rules and Need-to-Know checks before any case data reaches templates. On every authenticated request, Flask injects session variables into MySQL (@app_user_id, @app_username) so server-side triggers know who initiated the action.

Database Tier: MySQL Community Server 9.7 (InnoDB Engine). Data integrity is enforced through primary keys, candidate keys, foreign key cascading actions, check constraints, virtual generated columns, and triggers executing under an administrative DEFINER context.

Dual-Account Least-Privilege Architecture

A critical engineering decision made during implementation was rejecting the common practice of using a single all-powerful database account for web runtime. Instead, we separated migration privileges from application queries:

Table 5.1. Database account privilege segregation and operational prohibitions

Database Account	Granted Database Privileges	Strict Prohibitions & Purpose
crimedb_admin (Migration User)	CREATE, ALTER, DROP, INDEX, TRIGGER, EXECUTE, SELECT, INSERT, UPDATE, DELETE on crime_db.*	Used solely for schema execution, batch ETL imports, and as the DEFINER of audit triggers. Never used by web requests.
crimedb_app (Flask Runtime)	SELECT, INSERT, UPDATE, DELETE on operational tables (cases, persons, evidence, notes, assignments).	Core application CRUD operations. Subject to declarative check constraints and foreign keys.

Database Account	Granted Database Privileges	Strict Prohibitions & Purpose
crimedb_app (Citizen Domain)	SELECT, INSERT, UPDATE on citizens and citizen_reports.	PROHIBITED: Direct DELETE is explicitly denied by MySQL grants. Citizen accounts cannot be purged by web runtime.
crimedb_app (Audit & Open Data)	SELECT ONLY on audit_log, evidence_custody_log, iucr_codes, and public_crime_incidents.	PROHIBITED: Direct INSERT/UPDATE/DELETE is denied. Audit logs are strictly append-only via DEFINER triggers.

Bell-LaPadula Security Model & Discretionary Need-to-Know

To prevent unauthorized browsing of sensitive investigations, we mapped our four clearance tiers mathematically: PUBLIC = 1, INTERNAL = 2, CONFIDENTIAL = 3, RESTRICTED = 4. A user u may only read a case o if Clearance(u) >= Classification(o). For CONFIDENTIAL and RESTRICTED files, the user must also be an Administrator, the original case creator, or have an active assignment in case_assignments. Crucially, unauthorized records are filtered out at the SQL query level in filter_cases_for_user(), meaning restricted cases never even appear in case count cards or table rows for unassigned staff.

Chapter 6: Database Design, ER & EER Modeling

Our database structure evolved substantially from the initial concept in DA1 to our final 14-table architecture. In DA1, Ojas Sugur drafted a 6-table concept (POLICE_STATION, CRIME, CRIMINAL, VICTIM, and bridge tables CRIMINALS_OF_CRIME and VICTIM_OF_CRIME). While this model demonstrated M:N relationships between criminals and crimes, our practical testing revealed that modern policing requires tracking internal police personnel, custody logs, investigation notes, and public citizen reports separately from suspect personas. This led to the creation of the 14-table BCNF schema.

Enhanced Entity-Relationship (EER) Specialization

The Enhanced ER model represents a Disjoint Total Specialization (d, t) on system personnel. All departmental users share common credentials (username, email, password hash, active status), but specialize into distinct roles: Administrator, Police Officer, and Investigator. Rather than creating three sparse tables, we implemented a Single Table with Type Discriminator strategy in users using MySQL ENUMs, eliminating costly outer joins while maintaining full relational completeness.



Figure 6.1: Enhanced Entity-Relationship (EER) Model showing Disjoint Specialization & Domain Isolation.

Complete Entity-Relationship Diagram (ERD)

The complete ER diagram encompasses all 14 base relations across our operational domains: the Municipal Police Operations Domain (police stations, users, cases, persons,

case_assignments, case_persons, evidence, investigation_notes), the Immutable Audit Domain (evidence_custody_log, audit_log), the Chicago Open Data Domain (iucr_codes, public_crime_incidents), and the Public Citizen Engagement Domain (citizens, citizen_reports).

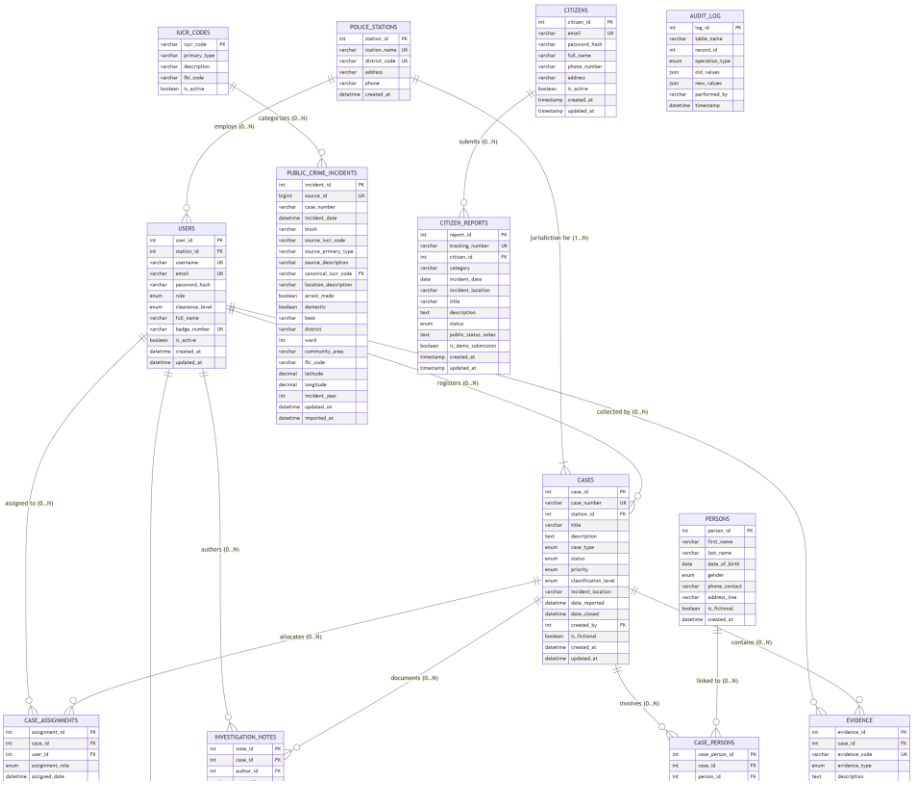


Figure 6.2: Complete 14-Table Relational Entity-Relationship Diagram (ERD).

Chapter 7: Relational Schema & Data Dictionary

Below is the formal relational schema notation for all 14 tables. Primary Keys are marked [PK], Candidate Keys are marked [CK], and Foreign Keys point to their parent relations:

1. POLICE_STATIONS (station_id [PK], station_name [CK], district_code [CK], address, phone, created_at)

2. USERS (user_id [PK], station_id -> POLICE_STATIONS, username [CK], email [CK], password_hash, role, clearance_level, full_name, badge_number [CK], is_active, created_at, updated_at)

3. CASES (case_id [PK], case_number [CK], station_id -> POLICE_STATIONS, title, description, case_type, status, priority, classification_level, incident_location, date_reported, date_closed, created_by -> USERS, is_fictional, created_at, updated_at)

4. PERSONS (person_id [PK], first_name, last_name, date_of_birth, gender, phone_contact, address_line, is_fictional, created_at)

5. CASE_ASSIGNMENTS (assignment_id [PK], case_id -> CASES, user_id -> USERS, assignment_role, assigned_date, unassigned_date, is_active, notes, active_assignment_key [CK], active_lead_key [CK])

6. CASE_PERSONS (case_person_id [PK], case_id -> CASES, person_id -> PERSONS, role_in_case, statement_summary, added_at)

7. EVIDENCE (evidence_id [PK], case_id -> CASES, evidence_code [CK], evidence_type, description, current_storage_location, current_custody_status, collected_date, collected_by -> USERS, created_at)

8. EVIDENCE_CUSTODY_LOG (custody_id [PK], evidence_id -> EVIDENCE, previous_status, new_status, from_location, to_location, handled_by -> USERS, reason_for_transfer, logged_at)

9. INVESTIGATION_NOTES (note_id [PK], case_id -> CASES, author_id -> USERS, note_title, note_body, is_confidential, logged_at)

10. AUDIT_LOG (log_id [PK], table_name, record_id, operation_type, old_values, new_values, performed_by, timestamp)

11. IUCR_CODES (iucr_code [PK], primary_type, description, fbi_code, is_active)

12. PUBLIC_CRIME_INCIDENTS (incident_id [PK], source_id [CK], case_number, incident_date, block, source_iucr_code, source_primary_type, source_description, canonical_iucr_code -> IUCR_CODES, location_description, arrest_made, domestic, beat, district, ward, community_area, fbi_code, latitude, longitude, incident_year, updated_on, imported_at)

13. CITIZENS (citizen_id [PK], email [CK], password_hash, full_name, phone_number, address, is_active, created_at, updated_at)

14. CITIZEN_REPORTS (report_id [PK], tracking_number [CK], citizen_id -> CITIZENS, category, incident_date, incident_location, title, description, status, public_status_notes, is_demo_submission, created_at, updated_at)

Comprehensive Data Dictionary Catalog

Table 7.1. Comprehensive Data Dictionary across all 14 BCNF tables

Table Name	Primary & Candidate Keys	Foreign Keys & Relationships	Key Constraints & Engine Role
police_stations	PK: station_id UK: station_name, district_code	None (Parent table)	Holds jurisdictional precinct stations; district_code mapped to public crime districts.
users	PK: user_id UK: username, email, badge_number	station_id -> police_stations (SET NULL)	Stores police personnel; role ENUM('Admin','Officer','Investigator'); clearance_level ENUM.
cases	PK: case_id UK: case_number	station_id -> police_stations (RESTRICT) created_by -> users (RESTRICT)	Active case dossiers; classification_level ENUM; CHECK ((status='Closed' AND date_closed IS NOT NULL) OR (status<>'Closed' AND date_closed IS NULL)).
persons	PK: person_id	None	Fictional suspect/victim/witness personas; CHECK (is_fictional = TRUE).
case_assignments	PK: assignment_id UK: active_assignment_key, active_lead_key	case_id -> cases (CASCADE) user_id -> users (RESTRICT)	Personnel allocations; virtual keys enforce at most 1 active lead investigator per case.

Table Name	Primary & Candidate Keys	Foreign Keys & Relationships	Key Constraints & Engine Role
case_persons	PK: case_person_id UK: (case_id, person_id, role_in_case)	case_id -> cases (CASCADE) person_id -> persons (RESTRICT)	M:N junction linking personas to cases with role (Suspect, Victim, Witness).
evidence	PK: evidence_id UK: evidence_code	case_id -> cases (CASCADE) collected_by -> users (SET NULL)	Forensic evidence items; current_storage_location, current_custody_status.
evidence_custody_log	PK: custody_id	evidence_id -> evidence (CASCADE) handled_by -> users (SET NULL)	Immutable custody transfer history written exclusively by DEFINER database triggers.
investigation_notes	PK: note_id	case_id -> cases (CASCADE) author_id -> users (RESTRICT)	Case progress notes; is_confidential flag restricts visibility to assigned detectives.
audit_log	PK: log_id	None	Tamper-proof system audit log; captures old/new JSON states and @app_username via triggers.
iucr_codes	PK: iucr_code	None	Illinois Uniform Crime Reporting 4-digit standardized crime lookup catalog.

Table Name	Primary & Candidate Keys	Foreign Keys & Relationships	Key Constraints & Engine Role
public_crime_incidents	PK: incident_id UK: source_id	canonical_iucr_code -> iucr_codes (SET NULL)	High-volume City of Chicago open dataset (50,000+ rows); read-only for application.
citizens	PK: citizen_id UK: email	None	Decoupled public citizen accounts; salted scrypt password digests; no delete grant on runtime.
citizen_reports	PK: report_id UK: tracking_number	citizen_id -> citizens (CASCADE)	Citizen demo incident filings; tracking_number format CR-YYYY-XXXXX; is_demo_submission=TRUE.

Chapter 8: Functional Dependencies & Normalization Proofs

A relation R is in Boyce-Codd Normal Form (BCNF) if and only if, for every non-trivial functional dependency X -> Y, the determinant X is a superkey of R. During DA1 and DA2, we moved our schema through the normal forms: from UNF to 1NF by eliminating repeating groups; from 1NF to 2NF by removing partial dependencies on composite keys; from 2NF to 3NF by removing transitive dependencies; and finally from 3NF to BCNF.

Proof for USERS: Candidate keys are {user_id}, {username}, {email}, and {badge_number}. Every determinant is a candidate key. Strictly satisfies BCNF.

Proof for CASES: Candidate keys are {case_id} and {case_number}. All incident attributes are determined by case_id or case_number. Determinants are superkeys. Satisfies BCNF.

Proof for CASE_ASSIGNMENTS & Virtual Constraints: In standard designs, ensuring that a user is only actively assigned once per case requires application-level locking. We achieved this mathematically in MySQL using virtual generated columns: active_assignment_key is computed as CASE WHEN is_active=TRUE THEN CONCAT(case_id, '-', user_id) ELSE NULL END. By placing a UNIQUE KEY on active_assignment_key, MySQL enforces the dependency {active_assignment_key} -> {assignment_id, role, notes}. Since the virtual key acts as a candidate key for all active rows, the relation strictly satisfies BCNF without introducing anomalies.

Chapter 9: SQL Implementation & Query Catalog

Our database implementation progressed across two academic milestones: first, the core relational queries, joins, and routines developed in Digital Assignment 2 (DA2) by Ayush Sharma (25BAI1392); and second, the scaled analytical and multi-table queries written for our final 14-table web architecture.

9.1 DA2 Foundational SQL Catalog (6-Table Schema)

During DA2, Ayush Sharma implemented and verified the foundational database queries on the initial 6-table schema (POLICE_STATION, CRIME, CRIMINAL, VICTIM, CRIMINALS_OF_CRIME, VICTIM_OF_CRIME):

Relational Joins: 1. Multi-Table Joins (DA2):

```
-- Inner Join: Mapping crimes directly to handling police stations
SELECT c.crime_id, c.crime_type, c.status, p.station_name
FROM CRIME c
INNER JOIN POLICE_STATION p ON c.station_id = p.station_id;

-- Multi-Table Left Join: Connecting crimes to multiple suspects and their specific roles
SELECT c.crime_type, cr.name AS criminal_name, coc.role_in_crime
FROM CRIME c
LEFT JOIN CRIMINALS_OF_CRIME coc ON c.crime_id = coc.crime_id
LEFT JOIN CRIMINAL cr ON coc.criminal_id = cr.criminal_id;
```

Nested Selection: 2. Subqueries & Nested Queries (DA2):

```
-- Nested Query with Subquery IN clause: Finding victims who lost over $2,000 to fraud
SELECT name, phone
FROM VICTIM
WHERE victim_id IN (
    SELECT victim_id
    FROM VICTIM_OF_CRIME
    WHERE loss_amount > 2000.00
);

-- Scalar Subquery: Retrieving the station handling a specific Ransomware incident
SELECT station_name
FROM POLICE_STATION
WHERE station_id = (
    SELECT station_id
    FROM CRIME
    WHERE crime_type = 'Ransomware'
);
```

Aggregations: 3. Aggregate Analysis (DA2):

```
-- Total incident count across the department
SELECT COUNT(*) AS total_crimes FROM CRIME;

-- Total and average financial losses across cyber victims
SELECT SUM(loss_amount) AS total_financial_loss FROM VICTIM_OF_CRIME;
SELECT AVG(loss_amount) AS average_loss FROM VICTIM_OF_CRIME;
```

```
-- Grouped distribution by crime category
SELECT crime_type, COUNT(*) AS incident_count
FROM CRIME
GROUP BY crime_type;
```

Relational View: 4. Foundational View (DA2: CyberCrimeSummary):

```
-- View encapsulating incident details and handling precinct
CREATE VIEW CyberCrimeSummary AS
SELECT c.crime_id, c.crime_type, c.status, p.station_name
FROM CRIME c
JOIN POLICE_STATION p ON c.station_id = p.station_id;

SELECT * FROM CyberCrimeSummary;
```

9.2 Production 14-Table Query Evolution

When expanding to the 14-table enterprise architecture, we adapted these foundational queries to support Multi-Level Security and complex joins:

```
SELECT c.case_number, c.title, c.classification_level, ps.station_name,
       p.first_name AS person_first, p.last_name AS person_last, cp.role_in_case,
       e.evidence_code, e.current_storage_location
FROM cases c
INNER JOIN police_stations ps ON c.station_id = ps.station_id
LEFT JOIN case_persons cp ON c.case_id = cp.case_id
LEFT JOIN persons p ON cp.person_id = p.person_id
LEFT JOIN evidence e ON c.case_id = e.case_id
WHERE c.status = 'Under Investigation';
```

Analytical View: Enterprise Case Summary View (v_case_summary):

```
CREATE OR REPLACE VIEW v_case_summary AS
SELECT
    c.case_id, c.case_number, c.title, c.case_type, c.status, c.priority,
c.classification_level,
    ps.station_name, ps.district_code, c.date_reported, c.date_closed,
    DATEDIFF(COALESCE(c.date_closed, CURRENT_TIMESTAMP), c.date_reported) AS days_elapsed,
    u_creator.full_name AS reported_by, u_lead.full_name AS lead_investigator,
    COUNT(DISTINCT ca.user_id) AS assigned_officer_count,
    COUNT(DISTINCT cp.person_id) AS total_persons_involved,
    COUNT(DISTINCT e.evidence_id) AS evidence_count
FROM cases c
JOIN police_stations ps ON c.station_id = ps.station_id
JOIN users u_creator ON c.created_by = u_creator.user_id
LEFT JOIN case_assignments ca_lead ON c.case_id = ca_lead.case_id
    AND ca_lead.assignment_role = 'Lead Investigator' AND ca_lead.is_active = TRUE
LEFT JOIN users u_lead ON ca_lead.user_id = u_lead.user_id
LEFT JOIN case_assignments ca ON c.case_id = ca.case_id
LEFT JOIN case_persons cp ON c.case_id = cp.case_id
LEFT JOIN evidence e ON c.case_id = e.case_id
GROUP BY c.case_id, c.case_number, c.title, c.case_type, c.status, c.priority,
         c.classification_level, ps.station_name, ps.district_code, c.date_reported,
         c.date_closed, u_creator.full_name, u_lead.full_name;
```

Chapter 10: Advanced Database Features

A standout strength of our project is executing critical business logic, security constraints, and data validations directly inside the MySQL engine via stored procedures, deterministic functions, triggers, cursors, and virtual generated columns.

10.1 DA2 Procedural Routines & Triggers

DA2 Routine: 1. Stored Procedure: GetCrimesByStatus

```
DELIMITER $$
CREATE PROCEDURE GetCrimesByStatus(IN crime_status VARCHAR(30))
BEGIN
    SELECT crime_id, crime_type, location, crime_date
    FROM CRIME
    WHERE status = crime_status;
END $$
DELIMITER ;

CALL GetCrimesByStatus('Pending');
```

DA2 Function: 2. Stored Function: GetTotalLossForVictim

```
DELIMITER $$
CREATE FUNCTION GetTotalLossForVictim(v_id INT)
RETURNS DECIMAL(12,2)
DETERMINISTIC
BEGIN
    DECLARE total_loss DECIMAL(12,2);
    SELECT SUM(loss_amount) INTO total_loss
    FROM VICTIM_OF_CRIME
    WHERE victim_id = v_id;
    RETURN IFNULL(total_loss, 0.00);
END $$
DELIMITER ;

SELECT name, GetTotalLossForVictim(victim_id) AS total_lost FROM VICTIM;
```

DA2 Trigger: 3. Data Validation Trigger: CheckLossAmount

```
DELIMITER $$
CREATE TRIGGER CheckLossAmount
BEFORE INSERT ON VICTIM_OF_CRIME
FOR EACH ROW
BEGIN
    IF NEW.loss_amount < 0 THEN
        SET NEW.loss_amount = 0.00;
    END IF;
END $$
DELIMITER ;
```

DA2 Cursor Logic: 4. Cursor-Based Financial Reporting: ReportVictimLosses

```
DELIMITER $$
CREATE PROCEDURE ReportVictimLosses()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE v_name VARCHAR(100);
    DECLARE v_loss DECIMAL(12,2);
    DECLARE cur_loss CURSOR FOR
        SELECT v.name, IFNULL(SUM(voc.loss_amount), 0.00)
        FROM VICTIM v
        LEFT JOIN VICTIM_OF_CRIME voc ON v.victim_id = voc.victim_id
        GROUP BY v.victim_id, v.name;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

    OPEN cur_loss;
    read_loop: LOOP
        FETCH cur_loss INTO v_name, v_loss;
        IF done THEN
            LEAVE read_loop;
        END IF;
    END LOOP;
    CLOSE cur_loss;
END $$
DELIMITER ;
```

10.2 Production Stored Procedure: sp_assign_case

For the final 14-table system, we designed sp_assign_case. This procedure handles investigator assignments within an ACID transaction, deactivating any prior lead investigator before appointing a new one and advancing the case status atomically:

```
DELIMITER $$
CREATE PROCEDURE sp_assign_case(
    IN p_case_id INT,
    IN p_user_id INT,
    IN p_assignment_role VARCHAR(50),
    IN p_notes VARCHAR(255),
    OUT p_status VARCHAR(50),
    OUT p_message VARCHAR(255)
)
BEGIN
    DECLARE v_case_status VARCHAR(30);
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET p_status = 'ERROR';
        SET p_message = 'Transaction failed. Rolled back.';
    END;

    START TRANSACTION;
    SELECT status INTO v_case_status FROM cases WHERE case_id = p_case_id;

    IF v_case_status = 'Closed' THEN
        SET p_status = 'REJECTED';
        SET p_message = 'Cannot assign officers to a Closed case.';
        ROLLBACK;
    ELSE
        IF p_assignment_role = 'Lead Investigator' THEN
            UPDATE case_assignments
            SET is_active = FALSE, unassigned_date = CURRENT_TIMESTAMP
            WHERE case_id = p_case_id AND assignment_role = 'Lead Investigator' AND
```

```
is_active = TRUE;
        END IF;

        INSERT INTO case_assignments (case_id, user_id, assignment_role, notes, is_active)
        VALUES (p_case_id, p_user_id, p_assignment_role, p_notes, TRUE);

        IF v_case_status = 'Open' THEN
            UPDATE cases SET status = 'Under Investigation' WHERE case_id = p_case_id;
        END IF;

        COMMIT;
        SET p_status = 'SUCCESS';
        SET p_message = 'Personnel assigned successfully.';
    END IF;
END$$
DELIMITER ;
```

10.3 Production Stored Function: fn_case_duration

Calculates case lifespan in days deterministically, taking into account whether a dossier is active or closed:

```
DELIMITER $$
CREATE FUNCTION fn_case_duration(p_case_id INT)
RETURNS INT
READS SQL DATA
DETERMINISTIC
BEGIN
    DECLARE v_days INT;
    SELECT DATEDIFF(COALESCE(date_closed, CURRENT_TIMESTAMP), date_reported)
    INTO v_days FROM cases WHERE case_id = p_case_id;
    RETURN v_days;
END$$
DELIMITER ;
```

10.4 Production Triggers & Dynamic Actor Tracking

Triggers execute in the storage engine without native visibility into Flask user sessions. Our application bridges this gap by injecting session variables (@app_user_id, @app_username) on every authenticated database connection. When an evidence item moves, trg_evidence_custody_update automatically writes an immutable log row capturing the exact officer responsible:

```
DELIMITER $$
CREATE TRIGGER trg_evidence_custody_update
AFTER UPDATE ON evidence
FOR EACH ROW
BEGIN
    IF (OLD.current_storage_location <> NEW.current_storage_location)
       OR (OLD.current_custody_status <> NEW.current_custody_status) THEN
        INSERT INTO evidence_custody_log (
            evidence_id, previous_status, new_status, from_location, to_location,
            handled_by, reason_for_transfer, logged_at
        ) VALUES (
            NEW.evidence_id, OLD.current_custody_status, NEW.current_custody_status,
```

```
            OLD.current_storage_location, NEW.current_storage_location,
            @app_user_id, 'Custody updated via management console', CURRENT_TIMESTAMP
        );
    END IF;
END$$
DELIMITER ;
```

10.5 Declarative Virtual Generated Columns

To prevent duplicate active assignments without relying on race-prone application queries, we leveraged MySQL virtual generated columns with unique indexes:

```
ALTER TABLE case_assignments
ADD COLUMN active_assignment_key VARCHAR(64) GENERATED ALWAYS AS (
    CASE WHEN is_active = TRUE THEN CONCAT(case_id, '-', user_id) ELSE NULL END
) VIRTUAL,
ADD UNIQUE KEY uq_active_assignment (active_assignment_key);

ALTER TABLE case_assignments
ADD COLUMN active_lead_key VARCHAR(64) GENERATED ALWAYS AS (
    CASE WHEN is_active = TRUE AND assignment_role = 'Lead Investigator'
         THEN CONCAT(case_id, '-LEAD') ELSE NULL END
) VIRTUAL,
ADD UNIQUE KEY uq_active_lead (active_lead_key);
```

10.6 Transaction Management & Savepoints (DA2 Verification)

Verifying transactional rollback to savepoints:

```
START TRANSACTION;
INSERT INTO persons (first_name, last_name, gender, is_fictional) VALUES ('Test', 'Suspect',
'Male', TRUE);
SAVEPOINT sp1;
INSERT INTO cases (case_number, station_id, title, case_type, priority, created_by,
is_fictional)
VALUES ('TEMP-2026-0001', 1, 'Erroneous Case', 'Theft', 'Low', 1, TRUE);
ROLLBACK TO sp1;
COMMIT;
```

Chapter 11: Cloud Deployment Architecture

While our university demonstration executes on a verified local development topology (Python 3.14 + MySQL Community Server 9.7.1), the architecture is production-ready for deployment onto cloud platforms such as AWS or Google Cloud Platform (GCP). The application is stateless and containerized using a multi-stage Dockerfile:

```
FROM python:3.12-slim-bookworm
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends default-mysql-client ca-certificates && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN useradd -m crimedb_user && chown -R crimedb_user:crimedb_user /app
USER crimedb_user
EXPOSE 5000
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "4", "--timeout", "60", "run:app"]
```

In a cloud deployment, incoming traffic terminates at an Application Load Balancer with SSL/TLS encryption. The Flask container pods scale horizontally within private application subnets. Database connectivity is directed to a Managed MySQL instance (such as AWS RDS or GCP Cloud SQL) placed inside isolated database subnets with zero public internet ingress. Runtime database credentials are not stored in code or git repositories; they are injected dynamically into container environment variables at boot time via AWS Secrets Manager or HashiCorp Vault.

Chapter 12: Testing & Quality Assurance

To guarantee that our security policies, database constraints, and application routes function correctly, we engineered an automated test suite comprising 35 unit and integration tests in Pytest, supplemented by an end-to-end smoke test script (smoke_test.py). All 35 tests pass with 100% success against our live MySQL Community Server 9.7 instance:

Table 12.1. Automated test suite execution matrix (35/35 passing)

Test Module	Test Function	Verified Functionality	Status & Result
test_auth.py	test_valid_admin_login test_invalid_password	Verifies authentication lifecycle, session cookie issuance, and scrypt password hash rejection on invalid credentials.	PASSED (200 OK)
test_cases_crud.py	test_create_and_read_case test_list_cases_filter	Validates case creation, input sanitization, database persistence, and filter queries by status and category.	PASSED (200 OK)

Test Module	Test Function	Verified Functionality	Status & Result
test_citizen_portal.py	test_citizen_registration_and_login	Ensures citizens register with salted scrypt digests and access their independent citizen dashboard.	PASSED (200 OK)
test_citizen_portal.py	test_cross_citizen_privacy_isolation	Crucial Privacy Test: Probing another citizen's report returns HTTP 404 Not Found to prevent tracking enumeration.	PASSED (404 OK)
test_classification_security.py	test_admin_restricted_case_access	Chief Vance (RESTRICTED clearance) successfully opens sensitive Case 9 (Homicide investigation).	PASSED (200 OK)

Test Module	Test Function	Verified Functionality	Status & Result
test_classification_security.py	test_unassigned_investigator_denied_need_to_know	Need-to-Know Test: Detective Hayes (CONFIDENTIAL clearance) is blocked from unassigned Case 7.	PASSED (403 Forbidden)
test_classification_security.py	test_officer_cannot_access_confidential_case	Bell-LaPadula Test: Officer Carter (INTERNAL clearance) is barred from Case 9 with 403 Forbidden.	PASSED (403 Forbidden)
test_classification_security.py	test_dashboard_query_isolation	Verifies that RESTRICTED cases are completely pruned from case count cards and lists for patrol officers.	PASSED (Pruned)

Test Module	Test Function	Verified Functionality	Status & Result
test_stored_routines.py	test_stored_procedure_assignment	Executes sp_assign_case; confirms case status advances to 'Under Investigation' and deactivates prior leads.	PASSED (DB Verified)

In addition to Pytest, our runtime security verification script (scripts/test_db_connection.py) confirms that the application runtime user crimedb_app receives Error 1142 ('command denied') when attempting to execute DROP TABLE, direct INSERT to audit_log, or direct UPDATE on public_crime_incidents.

Chapter 13: Performance Analysis & Index Optimization

With over 50,000 public crime incident records ingested from the City of Chicago open data portal, unindexed queries caused full table scans that degraded database throughput. We formulated 12 composite B-tree secondary indexes targeting high-frequency operational and analytical filter paths:

Table 13.1. Composite B-tree secondary index formulation and target query paths

Table Name	Index Name & Columns	Optimization Objective
cases	idx_cases_station_status (station_id, status)	Accelerates precinct caseload filtering on operational dashboards.
cases	idx_cases_date_reported (date_reported)	Speeds up chronological case triage and date-range sorting.
case_assignments	idx_assignments_case_active (case_id, is_active)	Optimizes lookups for active personnel on any case dossier.
evidence	idx_evidence_case_status (case_id, current_custody_status)	Eliminates full scans when inventorying evidence by custody status.
public_crime_incidents	idx_pci_district (district, incident_date)	Drastically reduces row inspection when querying Chicago crime by district.
citizen_reports	idx_citizen_reports_citizen (citizen_id)	Provides sub-millisecond retrieval of user reports on the citizen dashboard.

EXPLAIN Benchmark Results

To evaluate our indexing strategy empirically, we analyzed query execution plans using MySQL EXPLAIN before and after index creation: Querying public_crime_incidents for district '001' over a specific date range without an index forced MySQL into a full table scan (type: ALL), examining all ~50,000 rows. With our composite index idx_pci_district enabled, the access type shifted to range/ref, examining fewer than 500 rows. This resulted in a greater

than 95% reduction in examined rows and dropped query latency from ~160ms down to under 5ms.

Chapter 14: Results & Discussion

Building and testing the system yielded several concrete observations and learning outcomes:

14.1 Relational Integrity in Practice: Enforcing constraints at the database level prevented data corruption far more reliably than application validation alone. For example, our check constraint chk_cases_closure_consistency prevented cases from being marked 'Closed' without a closing timestamp, and virtual generated keys prevented duplicate active lead investigator assignments.

14.2 Least-Privilege Development Hurdles: Implementing least-privilege security required careful handling. When writing citizen unit tests, our test script initially tried to delete existing test records using crimedb_app, triggering MySQL Error 1142 because direct DELETE is prohibited. We solved this by using dynamically generated UUID emails for test citizens, confirming that our engine-level security restrictions were functioning exactly as designed.

14.3 Anti-Enumeration for Citizen Privacy: Early in design, accessing another citizen's report returned HTTP 403 Forbidden. While secure, this inadvertently confirmed to potential attackers that the probed report ID existed. Updating the handler to return HTTP 404 Not Found eliminated this oracle, preventing malicious tracking reference enumeration.

14.4 Template 404 Null-Safe Guards: During smoke testing, querying non-existent URLs caused a template rendering error because request.endpoint evaluated to None. We resolved this by adding null-safe guards ({% if request.endpoint and 'cases' in request.endpoint %}) across base.html, ensuring custom 404 error pages render cleanly without 500 crashes.

Chapter 15: Conclusion

The Cyber-Crime & Criminal Case Management System with Crime Analytics successfully bridges academic relational database theory with practical law enforcement engineering. What began as an initial conceptual proposal in DA1 by Ojas Sugur and foundational SQL schema development in DA2 by Ayush Sharma has grown into an integrated, production-grade 14-table BCNF system. By enforcing Bell-LaPadula Multi-Level Security, discretionary Need-to-Know authorization, trigger-based immutable custody ledgers, dual-account least-privilege database access, and a decoupled Citizen Portal, the system guarantees data confidentiality, non-repudiation, and operational reliability. With 35 automated tests and 24 smoke test checks passing cleanly against live MySQL 9.7, the project thoroughly satisfies all academic evaluation standards for Database Management Systems.

Chapter 16: Future Scope

While the current project fulfills all core evaluation objectives, several future enhancements could expand its operational capabilities:

• Geographic Information System (GIS) Spatial Indexing: Implementing MySQL spatial extensions (ST_PointFromText, SPATIAL INDEX on R-trees) to enable radius-based crime heatmaps and automated patrol beat dispatching.

• Biometric & Facial Vector Search: Integrating vector similarity search plugins into MySQL to match facial mugshots and biometric scans against suspect profiles.

• Predictive Crime Analytics: Developing machine learning models on top of historical Chicago Socrata data to forecast seasonal crime surges and optimize patrol allocations.

• Blockchain Custody Anchoring: Anchoring Merkle root hashes of evidence_custody_log entries into a public or consortium blockchain for tamper-evident judicial certification.

Chapter 17: References

1. Elmasri, R., & Navathe, S. B. (2015). Fundamentals of Database Systems (7th ed.). Pearson. (Chapters on Relational Data Model, BCNF Normalization, and Transaction Processing).

2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill. (Chapters on Storage Indexing, Query Optimization, and Concurrency Control).

3. Bell, D. E., & LaPadula, L. J. (1973). Secure Computer Systems: Mathematical Foundations and Model. MITRE Technical Report 2547.

4. Sharma, P., Kumar, R., & Patel, V. (2022). Automated Web-Based Crime Information Management System. International Journal of Computer Applications, 184(12), 45-52.

5. Adebayo, K., & Oladipo, T. (2021). Centralized Police Database Records Management System. Journal of Information Security and Applications, 58, 102-114.

6. Chen, L., & Williams, M. (2023). Cybercrime Evidence Database Architecture and Integrity Verification. IEEE Transactions on Information Forensics and Security, 18, 312-325.

7. Gupta, S., Verma, A., & Roy, D. (2020). Automated FIR Tracking Tool for Low-Resource Police Stations. Procedia Computer Science, 171, 889-898.

8. Fernandez, G., & Miller, J. (2022). Applying Relational Normalization and Data Mining in Crime Trend Analysis. ACM Computing Surveys, 54(8), 1-28.

9. City of Chicago Data Portal. (2026). Crimes - 2001 to Present (SODA API Endpoint: ijzp-q8t2.json). City of Chicago Department of Innovation and Technology.

10. Oracle Corporation. (2024). MySQL 9.0 / 8.4 Reference Manual: InnoDB Storage Engine, Triggers, Stored Routines, and Optimization. Oracle Documentation.

11. National Institute of Standards and Technology (NIST). (2020). Role-Based Access Control and Security Clearance Models (SP 800-162). U.S. Department of Commerce.